

Relazione assignment 02

Manuel Buizo

December 9, 2024

Contents

1	Domain	2
1.1	User Story	2
1.1.1	User	2
1.1.2	Company	4
1.2	Use case diagram	6
1.2.1	Notes	6
1.2.2	Aspetti aggiuntivi	7
1.3	Domain Storytelling	8
1.4	Quality attributes scenarios	11
1.4.1	Common	11
1.4.2	User	12
1.4.3	Company	12
2	Domain model	13
2.0.1	Components e Connectors	14
3	Ubiquitous language	15
3.1	User Repository	15
3.2	E-bike Repository	15
3.3	User Application	15
3.4	Company Application	15
4	Bounded Context	16
5	Tecnologie utilizzate	17
5.1	Application-level Patterns	17
5.1.1	API Gateway Pattern	17
5.1.2	Service Discovery Pattern	18
5.2	Deployment Pattern	20
5.2.1		20
5.3	Observability Patterns	21
5.4	Runtime Configuration Patterns	22
5.5	Testing patterns	24

1 Domain

Si vuole realizzare un servizio di noleggio di e-bike, con un'applicazione dedicata per gli utenti e un'interfaccia gestionale per la compagnia che gestisce il servizio.

L'applicazione permette agli utenti di registrarsi, accedere al sistema, noleggiare e utilizzare le e-bike, e visualizzare il proprio credito residuo.

La compagnia può aggiungere nuove e-bike al sistema, monitorare lo stato delle batterie e ricaricare quelle scariche, oltre a monitorare e consultare in tempo reale i dati delle e-bike e degli utenti.

L'obiettivo è simulare il funzionamento di un servizio di noleggio di e-bike, con un focus sull'interazione tra utenti e applicazione, e tra la compagnia e l'interfaccia gestionale.

Di seguito sono riportate le user stories relativi agli utenti e alla compagnia, per riuscire ad identificare i requisiti del sistema.

1.1 User Story

1.1.1 User

User Story 1: As a user, I want to sign up, so that I can use e-bike hire service

Scenario 1: User sign up with same username of another user

Given: The user is not registered

When: The user sign up with the same username of another user

Then: The system shows an error message, the username is already registered

Scenario 2: User sign up with correct username

Given: The user is not registered

When: The user sign up with correct username

Then: The system register user, and user access to the service

Scenario 3: User sign up with empty username or password

Given: The user is not registered

When: The user sign up with empty username or password

Then: The system shows an error message, the username or password is empty

User Story 2: As a user, I want to sign in, so that I can use e-bike hire service

Scenario 1: User sign in with wrong username

Given: The user is not registered

When: The user sign in with wrong username

Then: The system shows an error message, the user is not registered

Scenario 2: User sign in with wrong password

Given: The user is registered

When: The user sign in with wrong password

Then: The system shows an error message, the password is wrong

Scenario 3: User sign in with empty username or password

Given: The user is registered

When: The user sign in with empty username or password

Then: The system give an error message, the username or password is empty

Scenario 4: User sign in with correct username and password

Given: The user is registered

When: The user sign in with correct username and password

Then: User access to the service

User Story 3: As a user, I want to add credit, so that I can hire e-bike

Scenario 1: User is logged in

Given: The user has sign in

Sub-scenario 1: User add negative credit

When: The user add negative credit

Then: The system shows an error message, the credit is negative

Sub-scenario 2: User add some credit

When: The user add some credit

Then: The system notify add credit

Sub-scenario 3: User add empty credit

When: The user add empty credit

Then: The system shows an error message, the credit is empty

User Story 4: As a user, I want to start ride e-bike, so that I can use it

Scenario 1: User is logged in

Given: The user has sign in

Sub-scenario 1: User start ride e-bike with no credit

Given: The user has no credit

When: The user start ride e-bike

Then: The system shows an error message, the user has no credit

Sub-scenario 2: User start ride e-bike that is already in use

Given: The e-bike is in use

When: The user start ride e-bike

Then: The system shows an error message, the e-bike is already in use by another user

Sub-scenario 3: User start ride e-bike with empty battery

Given: the e-bike is with empty battery

When: The user start ride e-bike

Then: The system shows an error message, the e-bike has no battery

Sub-scenario 4: User start ride e-bike

Given: The user has some credit and the e-bike is free and has battery

When: The user start ride e-bike

Then: The system starts the ride, notify the user, subtract the credit

And: change the state of the e-bike

User Story 5: As a user, I want to ride e-bike, so that I can move

Scenario 1: user is logged in and ride e-bike

Given: The user has sign in

And: The user ride e-bike

Sub-scenario 1: User has some credit

Given: The user has some credit
When: The user ride e-bike
Then: The system subtract the credit, notify the user

Sub-scenario 2: User finished credit

Given: The user has no credit
When: The user ride e-bike
Then: The system stops the ride, notify the user and free the e-bike

User Story 6: As a user, I want to stop ride e-bike, so that I can end the ride

Scenario 1: User stop ride e-bike

Given: The user has sign in
When: The user ride e-bike
Then: The system stops the ride, notify the user and free the e-bike

User Story 7: As a user, I want to see my credit, so that I can monitor financial situation

Scenario 1: User see credit

Given: The user has sign in
When: The user open the application
Then: The system shows the credit

Scenario 2: User see payments

Given: The user has sign in
When: The user pay the unlock hire service
And: pay the ride service
Then: The system shows the residual credit

User Story 8: As a user, I want to see statistics of the e-bike, so that I can monitor the e-bike

Scenario 1: User see the hire e-bike

Given: The user has sign in and hired a e-bike
When: The user hired a e-bike
Then: The system shows statistics of the e-bike

Scenario 2: User see the e-bike in use

Given: The user has sign in and hired a e-bike
When: The user ride the e-bike
Then: The system update and show the statistics of the e-bike

1.1.2 Company

User Story 1: As a company, I want to access the application without the credentials, so that I can manage the service

Scenario 1: Company access the application

Given: The company has application
When: The company open the application
Then: The system show the dashboard

User Story 2: As a company, I want to register e-bike, so that I can manage the e-bikes

Scenario 1: Company register e-bike with same id of another e-bike

Given: The e-bike is not registered
When: The e-bike register with the same id of another bike
Then: The system shows an error message, the id is already registered

Scenario 2: Company register e-bike with correct id

Given: The e-bike is not registered
When: The e-bike register with correct id
Then: The system register the e-bike, and notify the company

User Story 3: As a company, I want to manage recharge e-bikes, so that I can keep the service running

Scenario 1: There are e-bikes with low battery

Given: The e-bikes has low battery
When: The company would recharge the e-bikes
Then: The system recharge the e-bike not use, change state and notify the company
And: For few second the e-bike is not available

Scenario 2: There aren't e-bikes with low battery

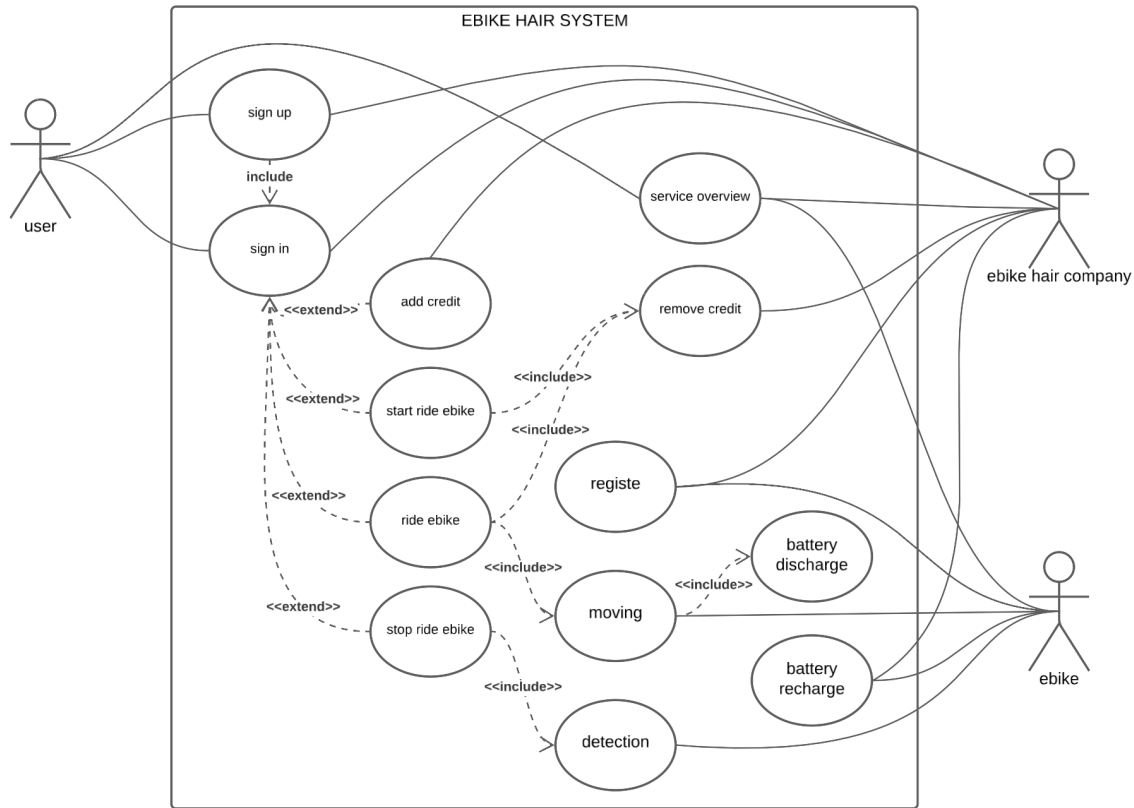
Given: The e-bikes has full battery
When: The company would recharge the e-bikes
Then: The system notify the company, there isn't any e-bike to recharge

User Story 4: As a company, I want to view general statistics, so that I can monitor the service

Scenario 1: Company view general statistics (base version)

Given: The company has access all data of users and e-bikes
When: The company open the application
Then: The system show users registered with a credit and that are using the e-bikes
And: e-bikes registered with position, battery and state,
And: company profit

1.2 Use case diagram



1.2.1 Notes

Considerate le dimensioni del sistema, si è deciso di non integrare servizi o attori esterni per l'autenticazione e la certificazione dei dati di accesso degli utenti, poiché non risultano pertinenti al contesto del progetto.

Tuttavia, in un'applicazione reale, sarebbe necessario includere tali attori, come provider di identità (ad esempio Gmail, Outlook) e sistemi di crittografia avanzata. Per quanto riguarda le e-bike, il loro comportamento è stato simulato mediante un'interfaccia che consente sia le operazioni di noleggio sia la simulazione del movimento. Questa soluzione è stata progettata per semplificare l'integrazione futura di funzionalità avanzate, come la geolocalizzazione e il monitoraggio in tempo reale, da implementare in e-bike operative.

Inoltre, non è stato incluso un provider di pagamento, come carte di credito o altre modalità, poiché non rilevante nel contesto del progetto. In un'applicazione reale, sarebbe essenziale implementare un sistema di pagamento sicuro e robusto, in grado di garantire affidabilità e protezione dei dati sensibili.

Infine come da user stories, la compagnia può accedere all'applicazione senza credenziali, in quanto è l'azienda che gestisce il servizio e quindi ha accesso a tutte le funzionalità dell'applicazione.

1.2.2 Aspetti aggiuntivi

Potrebbero essere integrate ulteriori funzionalità per migliorare il sistema di gestione delle e-bike. Tra queste:

- Prenotazione: l'introduzione di un sistema che consenta agli utenti di prenotare un'e-bike per un periodo di tempo specifico, garantendo così la disponibilità del mezzo al momento del noleggio.
- Segnalazione: la possibilità di segnalare eventuali malfunzionamenti o danni a un'e-bike direttamente tramite l'interfaccia utente, facilitando l'intervento tempestivo da parte dell'azienda di noleggio per manutenzione o sostituzione.
- Abbonamento: un'opzione per sottoscrivere abbonamenti che offrano tariffe agevolate per il noleggio delle e-bike, incentivando l'uso continuativo del servizio.

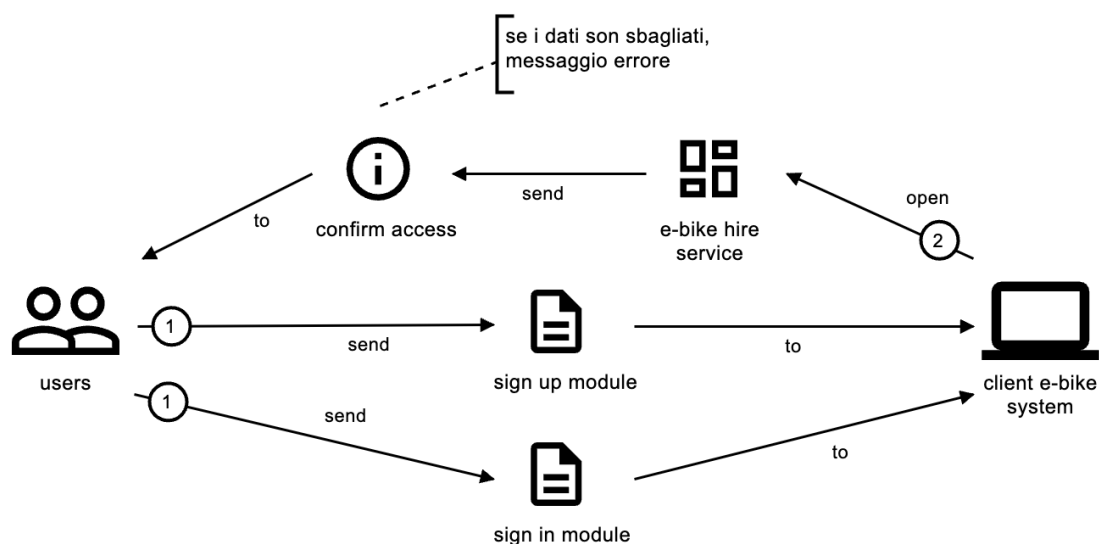
Queste funzionalità contribuirebbero ad arricchire l'esperienza utente e a ottimizzare la gestione operativa del sistema.

1.3 Domain Storytelling

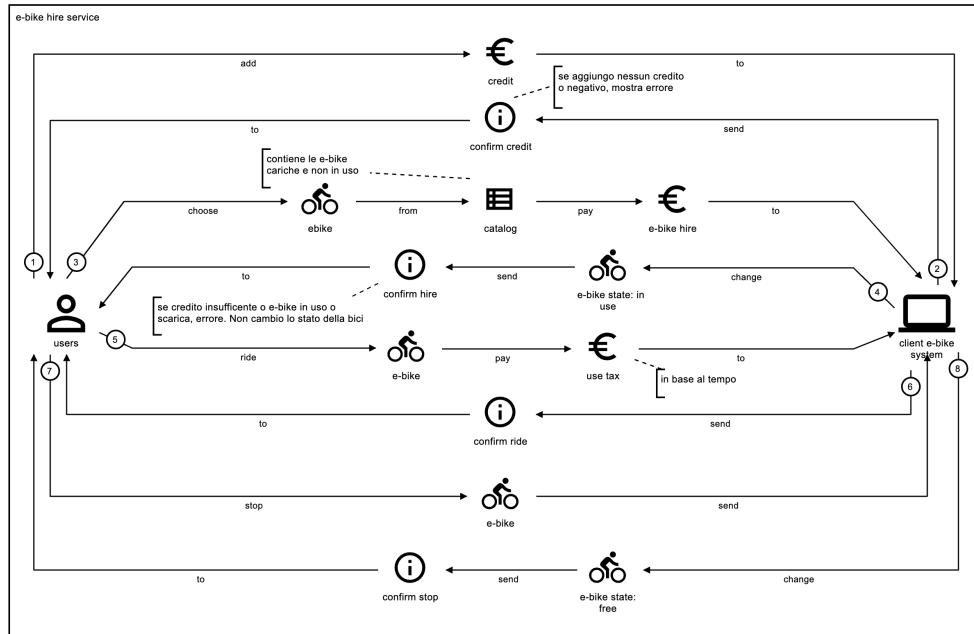
È stato adottato il metodo del *Domain Storytelling* per descrivere in dettaglio il processo di business della compagnia di noleggio e il flusso del noleggio di un'e-bike da parte dell'utente. Gli attori principali identificati sono:

- La compagnia, che gestisce il servizio.
- L'applicazione, che funge da intermediario tecnologico, sia per la compagnia che per l'utente.
- L'utente, che interagisce con l'app per noleggiare e restituire l'e-bike.

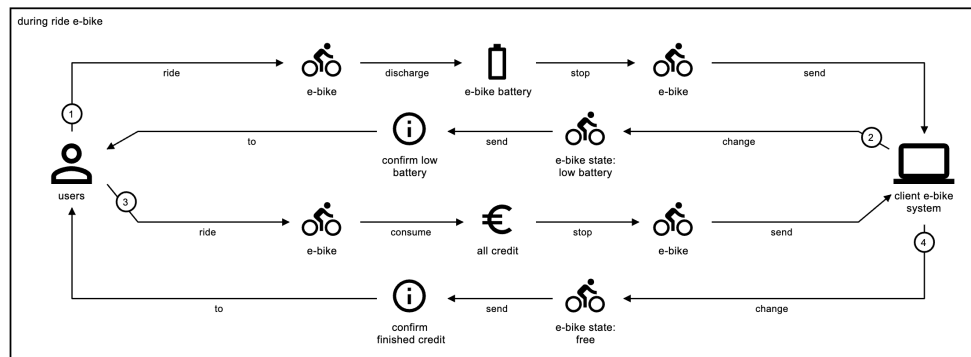
Le domain stories sono state sviluppate con il *domain expert* e gli sviluppatori per ottenere una visione condivisa del processo. Questo approccio ha permesso di descrivere le dinamiche operative, le regole di business e le interazioni tra gli attori. Di seguito sono riportate le domain stories relative al processo di noleggio.



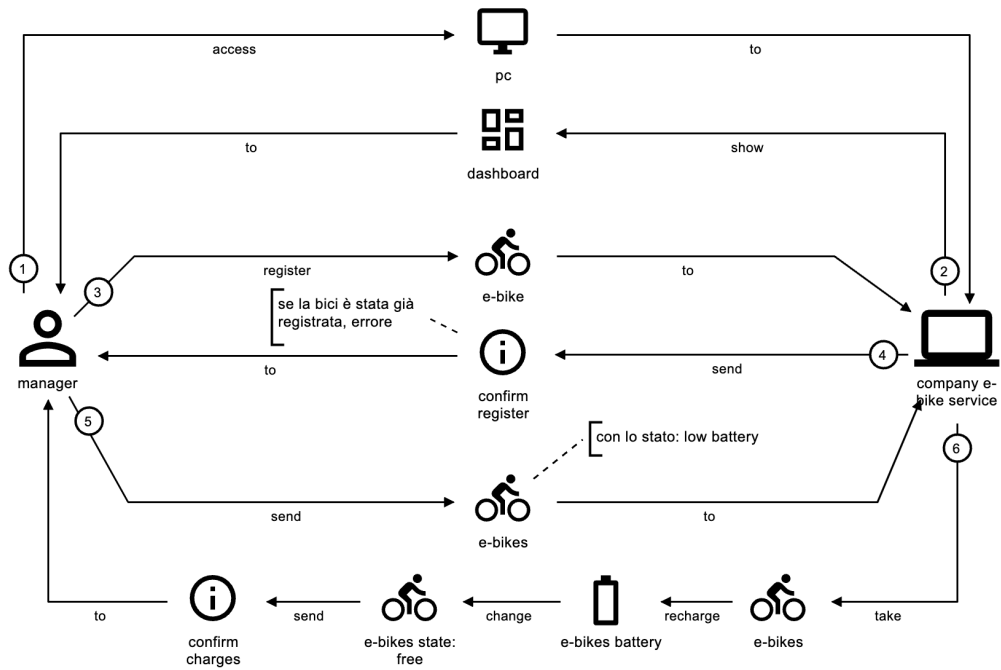
Questa *domain story* descrive il processo di registrazione di un utente all'interno dell'applicazione. L'utente inserisce i dati richiesti, tra cui username e password, e conferma la registrazione. Se l'username è già in uso, il sistema restituisce un messaggio di errore. Altrimenti, l'utente viene registrato e può accedere al servizio.



Questa *domain story* descrive il processo di noleggio di un'e-bike da parte dell'utente. Il processo inizia con l'utente che ricarica il proprio credito sull'app. Successivamente, avvia il noleggio dell'e-bike. Se l'e-bike è disponibile e il credito è sufficiente, il noleggio viene confermato con successo. Durante il noleggio, l'utente può utilizzare l'e-bike liberamente e, al termine, concludere il noleggio tramite l'app.



Questa *domain story* descrive il processo di utilizzo di un'e-bike durante il noleggio, evidenziando i punti chiave del flusso. Include il monitoraggio del consumo della batteria durante l'uso e prevede la conclusione automatica del noleggio in caso di batteria insufficiente. Inoltre, illustra come il sistema gestisce eventuali situazioni di credito insufficiente.



Questa *domain story* descrive il processo di gestione delle e-bike da parte della compagnia. Include la registrazione di nuove e-bike nel sistema, la verifica dello stato delle batterie e la ricarica delle e-bike scariche.

1.4 Quality attributes scenarios

Di seguito sono riportati gli scenari relativi ai quality attributes del sistema, come la *reliability*, la *correctness*, la *verifiability*, la *scalability*, ...

Per riuscire a garantire la qualità del sistema, è necessario definire e testare questi scenari, al fine di verificare che il sistema soddisfi i requisiti di qualità richiesti.

1.4.1 Common

Performance scenario: le interazioni col sistema sono completate entro 2 secondi

Source: 500 users, managers of company

Stimulus: use the hire e-bike system, use the monitoring bike system

Artifact: system

Environment: normal operation, overload operation

Response: notify action completed

Response measure: mean response time < 2s

Verifiability scenario: il sistema deve verificare la correttezza dei dati inseriti dall'utente e le azioni compiute

Source: users, managers of company

Stimulus: use the hire e-bike system, use the monitoring bike system

Artifact: system

Environment: normal operation, overload operation

Response: notify action completed

Response measure: no error

Reliability scenario: il sistema deve garantire il corretto funzionamento

Source: users, managers of company

Stimulus: use the hire e-bike system, use the monitoring bike system

Artifact: system

Environment: normal operation, overload operation

Response: do the action requested

Response measure: no error

Portability scenario: il sistema deve funzionare su dispositivi desktop sia mac, windows e linux

Source: users, managers of company

Stimulus: all system functionality

Artifact: system

Environment: normal operation, overload operation

Response: show the page

Response measure: no error

Modularity Scenario: il sistema deve essere diviso in moduli indipendenti

Source: users, managers of company

Stimulus: change the system

Artifact: system

Environment: normal operation, overload operation

Response: change the system

Response measure: mean response time < 3gg

1.4.2 User

Reusability scenario: l'applicazione della registrazione dell'utente può essere riutilizzato per la registrazione dei manager

Source: users register system

Stimulus: sign up, sign in

Artifact: application

Environment: normal operation, overload operation

Response: register the managers

Response measure: register the managers, no errors

Scalability scenario: l'applicazione degli utenti supporta l'aggiunta di nuove funzionalità

Source: users

Stimulus: e-bike book system, subscription system

Artifact: application

Environment: normal operation, overload operation

Response: add new functionality

Response measure: add new functionality, no errors

1.4.3 Company

Scalability scenario: l'applicazione dei manager supporta l'aggiunta di nuove funzionalità

Source: managers of company

Stimulus: add new data of e-bike, add register system

Artifact: application

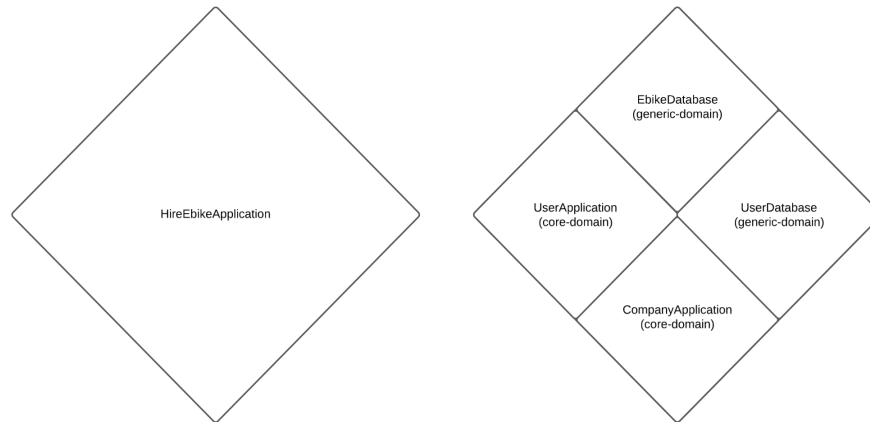
Environment: normal operation, overload operation

Response: add new functionality

Response measure: add new functionality, no errors

2 Domain model

Di seguito sono riportati il domain model e i sub-domain del sistema di noleggio delle e-bike.



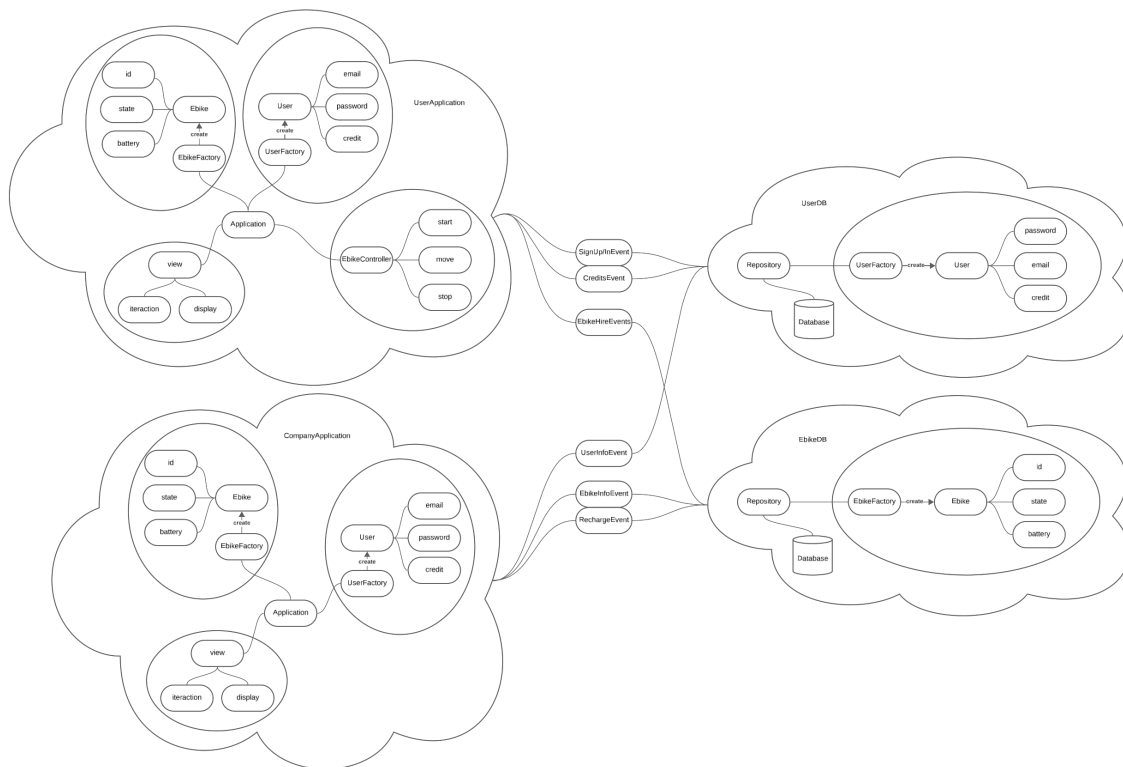
Aumentando la granularità del modello, si possono identificare gli elementi principali che compongono ciascun sub-domain:

UserDB: gestisce le informazioni degli utenti, come username, password e credito.

E-bikeDB: gestisce le informazioni delle e-bike, come id, stato e batteria.

UserApplication: gestisce le interazioni degli utenti con il sistema.

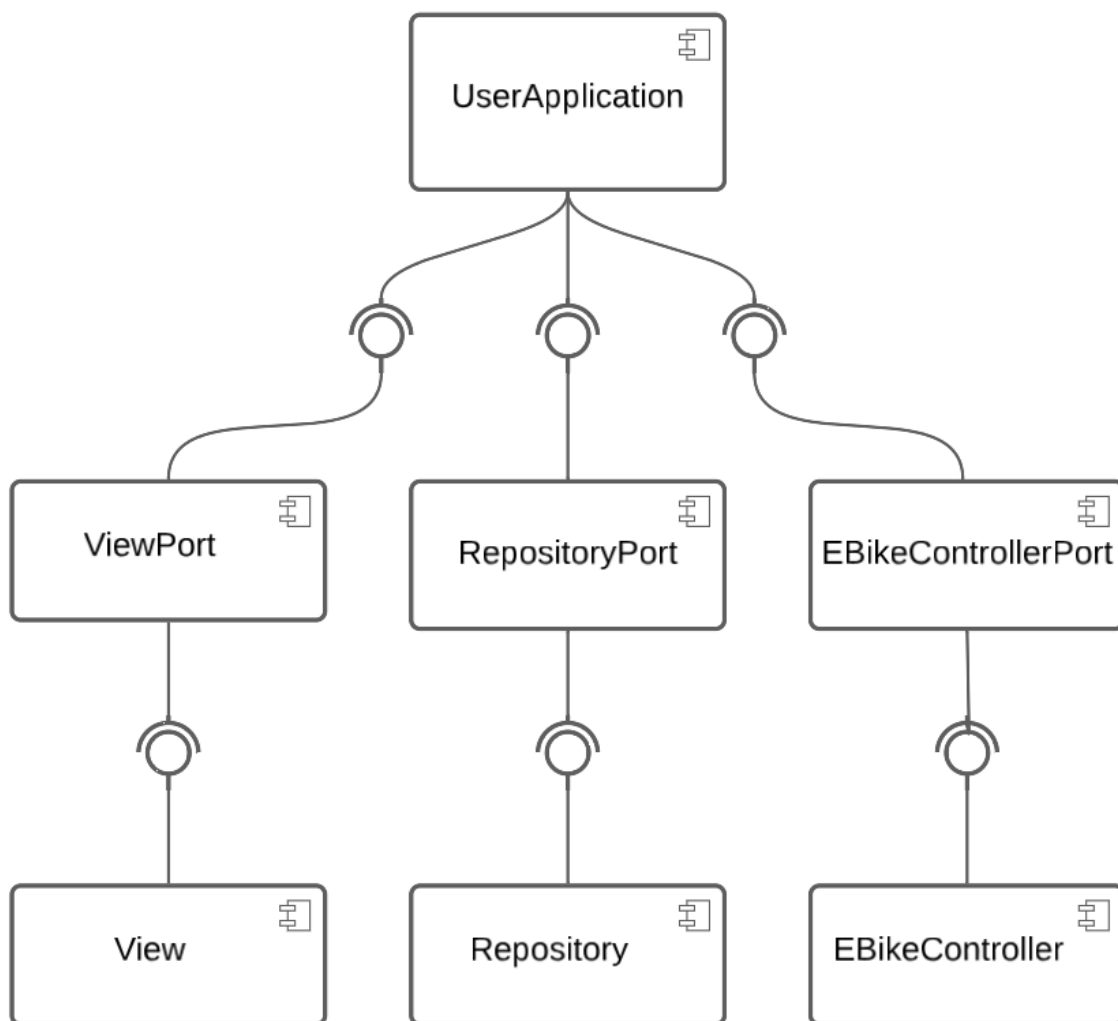
CompanyApplication: gestisce le interazioni della compagnia con il sistema.



2.0.1 Components e Connectors

L'architettura sia della UserRepository che della E-bikeRepository è basata su un'architettura a componenti e connettori. Data la loro semplicità e la loro funzionalità, sono stati progettati come componenti indipendenti. Hanno le entità principali e il livello applicativo che funge da repository per le entità.

Per quanto riguarda l'UserApplication è progettata come un'architettura a componenti e connettori, con un'interfaccia utente che interagisce con il sistema. Il livello applicativo gestisce le richieste degli utenti e le trasmette al repository per l'elaborazione. Ha tre porte che permettono l'interazione con l'utente, il repository e il sistema di movimento delle bici.



3 Ubiquitous language

In questa sezione vengono definiti i termini del ubiquitous language utilizzati nel sistema di noleggio delle e-bike. Questi termini sono stati identificati in collaborazione con il domain expert e gli sviluppatori per creare un vocabolario condiviso e chiaro per il progetto.

3.1 User Repository

- Username: nome utente utilizzato per accedere al sistema.
- Password: codice segreto utilizzato per accedere al sistema.
- Credit: saldo disponibile per il noleggio delle e-bike.

3.2 E-bike Repository

- Id: identificativo univoco assegnato a ciascuna e-bike.
- State: stato corrente dell'e-bike (libera, in uso, in manutenzione).
 - Free: l'e-bike è disponibile per il noleggio.
 - In use: l'e-bike è attualmente in uso da un utente.
 - Low battery: l'e-bike ha la batteria scarica e non è disponibile per il noleggio.
- Battery: livello di carica della batteria dell'e-bike.

3.3 User Application

- Sign up: processo di registrazione di un nuovo utente nel sistema.
- Sign in: processo di accesso di un utente al sistema.
- Add credit: aggiunta di credito al saldo dell'utente.
- Hire e-bike: noleggio di un'e-bike da parte dell'utente.
- Ride: utilizzo dell'e-bike durante il noleggio.
- Stop EBIke: conclusione del noleggio di un'e-bike da parte dell'utente.
- See credit: visualizzazione del saldo disponibile dell'utente.
- See statistics: visualizzazione delle statistiche dell'e-bike noleggiata.
- See payments: visualizzazione dei pagamenti effettuati dall'utente.

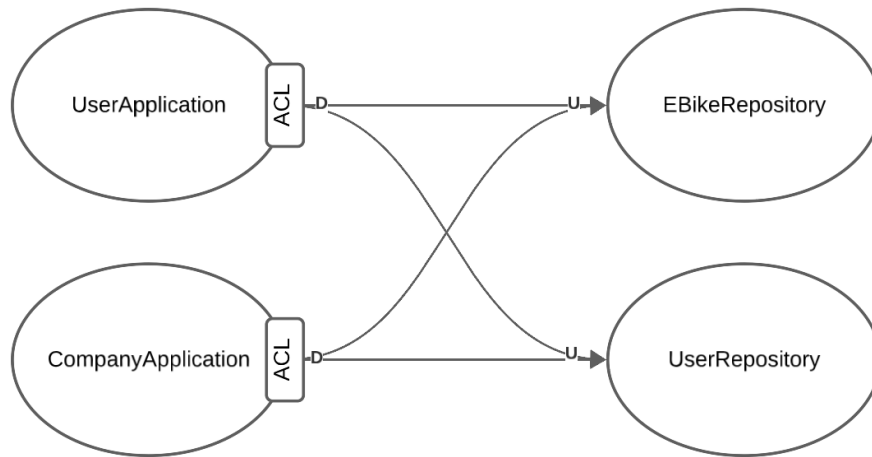
3.4 Company Application

- Register e-bike: registrazione di una nuova e-bike nel sistema.
- Recharge e-bike: ricarica di una e-bike con batteria scarica.
- View statistics: visualizzazione delle statistiche generali del servizio.

4 Bounded Context

Si identificano quattro *bounded context* all'interno del sistema di noleggio delle e-bike:

- User Context: gestisce le informazioni degli utenti, come username, password e credito.
- E-bike Context: gestisce le informazioni delle e-bike, come id, stato e batteria.
- User Application Context: gestisce le interazioni degli utenti con il sistema.
- Company Application Context: gestisce le interazioni della compagnia con il sistema.



Sono stati creati dei layer per i downstream al fine di separare le responsabilità, assicurando così coesione e bassa dipendenza tra i vari componenti. Questo approccio consente di mantenere distinti i tipi di dati delle applicazioni, evitando sovrapposizioni e garantendo coerenza e consistenza nelle informazioni.

5 Tecnologie utilizzate

Per lo sviluppo del sistema di noleggio delle e-bike, sono state utilizzate le seguenti tecnologie:

- Java: linguaggio di programmazione utilizzato per lo sviluppo del backend.
- Javalin: framework per la creazione di API RESTful in Java.
- OkHttp: libreria per le chiamate HTTP in Java.
- Consul: servizio di discovery e configuration per la gestione dei microservizi.
- Docker: piattaforma per la creazione, la distribuzione e l'esecuzione di container.
- Prometheus: sistema di monitoraggio e alerting.
- Cucumber: framework per il testing di accettazione.
- JUnit: framework per il testing unitario.
- PostMan: strumento per testare le API RESTful.
- Git: sistema di controllo di versione distribuito.

5.1 Application-level Patterns

5.1.1 API Gateway Pattern

Il pattern API Gateway è un'architettura utilizzata per centralizzare il punto di accesso a più microservizi. L'API Gateway funge da intermediario tra il client e i servizi backend, ricevendo le richieste, instradandole ai microservizi appropriati, e aggregando le risposte prima di restituirle al client. Questo pattern aiuta a semplificare la gestione delle API e a ridurre la complessità del sistema, centralizzando operazioni come l'autenticazione, la gestione dei permessi, il bilanciamento del carico e la gestione dei log.

Architettura e Tecnologie Utilizzate

L'API Gateway è stato sviluppato utilizzando le seguenti tecnologie:

- Javalin: Un framework web minimalista per Java, scelto per la sua leggerezza e facilità d'uso. Javalin consente di creare facilmente applicazioni web e API e si è rivelato ideale per implementare un API Gateway semplice e performante.
- OkHttp: Una libreria HTTP client, utilizzata per inviare richieste HTTP ai microservizi interni. OkHttp è stato scelto per la sua efficienza e affidabilità nella gestione delle comunicazioni tra l'API Gateway e i servizi backend.

Il sistema è configurato per gestire tutte le richieste in ingresso sulla porta 3000. L'API Gateway riceve le richieste da parte dei client e le inoltra ai microservizi appropriati tramite OkHttp. Successivamente, raccoglie le risposte dai microservizi e le restituisce ai client, garantendo un punto di accesso centralizzato e semplificato.

Vantaggi dell'API Gateway

L'adozione di un API Gateway porta diversi vantaggi:

1. Centralizzazione della gestione delle richieste: L'API Gateway centralizza la gestione delle richieste, riducendo il numero di punti di ingresso nel sistema e semplificando il routing delle richieste ai servizi corretti.
2. Astrazione dei microservizi: I client non devono preoccuparsi della posizione o dei dettagli di implementazione dei singoli microservizi. L'API Gateway si occupa di instradare le richieste in modo trasparente, migliorando l'interoperabilità tra i client e i microservizi.
3. Gestione della sicurezza: L'API Gateway può gestire l'autenticazione e l'autorizzazione in modo centralizzato, applicando politiche di sicurezza come il controllo degli accessi, riducendo la necessità di implementare queste logiche in ogni singolo microservizio.
4. Monitoraggio e logging centralizzati: Con un API Gateway, è più facile raccogliere e centralizzare i log e i dati di monitoraggio delle richieste, migliorando la capacità di diagnosticare problemi e ottimizzare le prestazioni del sistema.
5. Scalabilità e performance: L'API Gateway consente di implementare politiche di bilanciamento del carico, distribuendo le richieste in modo efficiente tra le varie istanze dei microservizi. Inoltre, centralizzare alcune operazioni come la gestione delle risposte riduce la complessità e ottimizza le performance complessive del sistema.
6. Trasparenza nella gestione dei dati: L'API Gateway può aggregare e trasformare le risposte provenienti dai microservizi prima di restituirle al client, semplificando il trattamento dei dati e migliorando l'interazione con i client.

5.1.2 Service Discovery Pattern

Il Discovery Service è un pattern architetturale utilizzato per gestire la registrazione e il ritrovamento dei microservizi in un sistema distribuito. In un ambiente di microservizi, i servizi sono spesso dinamici, scalabili orizzontalmente e possono cambiare frequentemente (ad esempio, con nuovi microservizi che vengono aggiunti o rimossi). Il pattern Discovery Service risolve il problema di trovare e comunicare tra i servizi senza che i client debbano conoscere gli indirizzi di rete specifici di ciascun servizio.

Un Service Discovery si occupa di mantenere un registro dei servizi disponibili, in modo che ogni microservizio possa essere localizzato da altri servizi in base al loro nome o categoria, riducendo la necessità di hardcodare gli indirizzi IP o i dettagli di configurazione.

Architettura e Tecnologie Utilizzate

Per implementare il Discovery Service, sono state utilizzate le seguenti tecnologie:

- Consul: Una potente soluzione di service discovery che consente di registrare i microservizi e recuperare le informazioni relative ai servizi attivi. Consul offre funzionalità di registrazione, consulenza (query) e monitoraggio dei servizi, il che lo rende ideale per implementare un sistema di discovery in ambienti di microservizi distribuiti.

- **Javalin:** Un framework web minimalista per Java, utilizzato per creare facilmente applicazioni web e API. Javalin è stato integrato con Consul per facilitare la registrazione e la ricerca dei microservizi, creando così un'interfaccia semplice per l'interazione con Consul.

L'implementazione consente ai microservizi di registrarsi automaticamente nel registro di Consul, mentre altri microservizi possono interrogare Consul per ottenere informazioni sui servizi disponibili, come indirizzi IP e porte. In questo modo, i servizi possono scoprire dinamicamente la posizione degli altri servizi senza necessità di configurazione manuale.

Funzionamento del Discovery Service

Il Discovery Service è stato sviluppato in modo tale che ogni microservizio possa registrarsi automaticamente con Consul al momento dell'avvio. Allo stesso tempo, altri servizi possono effettuare delle query a Consul per ottenere informazioni sui servizi registrati, come l'indirizzo e la porta del servizio richiesto.

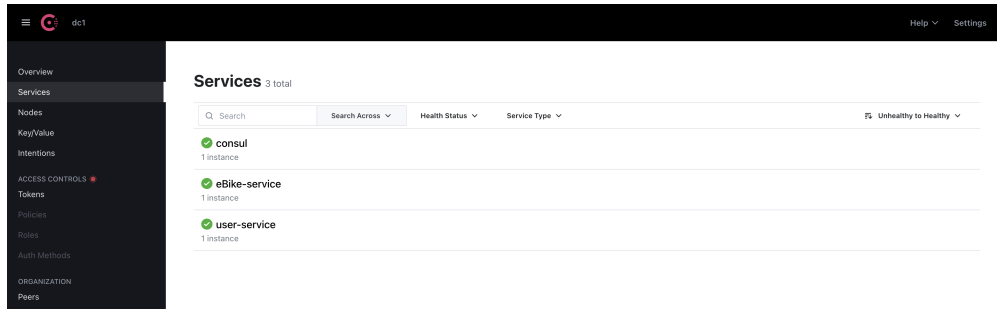
1. **Registrazione dei servizi:** Ogni microservizio, al momento del suo avvio, registra il proprio stato e le proprie informazioni (come l'IP e la porta) in Consul.
2. **Scoperta dei servizi:** Quando un altro microservizio ha bisogno di comunicare con un servizio registrato, effettua una richiesta a Consul per ottenere le informazioni necessarie (come l'indirizzo di rete del servizio) in modo dinamico.
3. **Monitoraggio della salute:** Consul esegue anche il monitoraggio dello stato dei servizi, garantendo che solo i servizi attivi siano inclusi nel registro e che le richieste siano indirizzate a servizi operativi.

Vantaggi del Pattern Discovery Service

L'adozione di un Discovery Service tramite Consul offre numerosi vantaggi:

1. **Scalabilità e flessibilità:** Il servizio di discovery consente di scalare facilmente il sistema aggiungendo o rimuovendo istanze di microservizi senza dover aggiornare manualmente la configurazione di rete o l'indirizzo dei servizi. Consul si occupa di aggiornare automaticamente il registro dei servizi.
2. **Indirizzamento dinamico:** Poiché i microservizi sono scoperti dinamicamente tramite Consul, non è necessario hardcodare indirizzi IP o porte nei client o nei servizi. I microservizi possono essere localizzati tramite nome e non sono legati a una posizione fissa.
3. **Alta disponibilità:** Consul è progettato per essere altamente disponibile e tollerante ai guasti. Se un'istanza di un microservizio fallisce, Consul aggiorna automaticamente il registro per rimuovere il servizio guasto, indirizzando le richieste verso istanze sane.
4. **Monitoraggio dello stato dei servizi:** Consul supporta il monitoraggio continuo della salute dei servizi, garantendo che solo i servizi attivi siano visibili per il discovery. Ciò riduce il rischio di comunicare con servizi non funzionanti.

5. Decoupling dei microservizi: I microservizi non hanno bisogno di sapere dove si trovano gli altri servizi, riducendo la complessità della configurazione e migliorando la modularità e la manutenibilità del sistema.
6. Semplificazione della gestione dei servizi: L'integrazione con Consul consente di avere una vista centralizzata di tutti i servizi in esecuzione, semplificando la gestione e il monitoraggio dei microservizi distribuiti.



5.2 Deployment Pattern

5.2.1 Deployment con Docker

In un ambiente software moderno, l'adozione di microservizi rappresenta un approccio architetturale essenziale per lo sviluppo di applicazioni scalabili e facilmente gestibili. Questo documento esplora il deployment pattern adottato per il deployment di un'architettura basata su microservizi, utilizzando Docker come tecnologia principale per la containerizzazione.

Architettura e Tecnologie Utilizzate

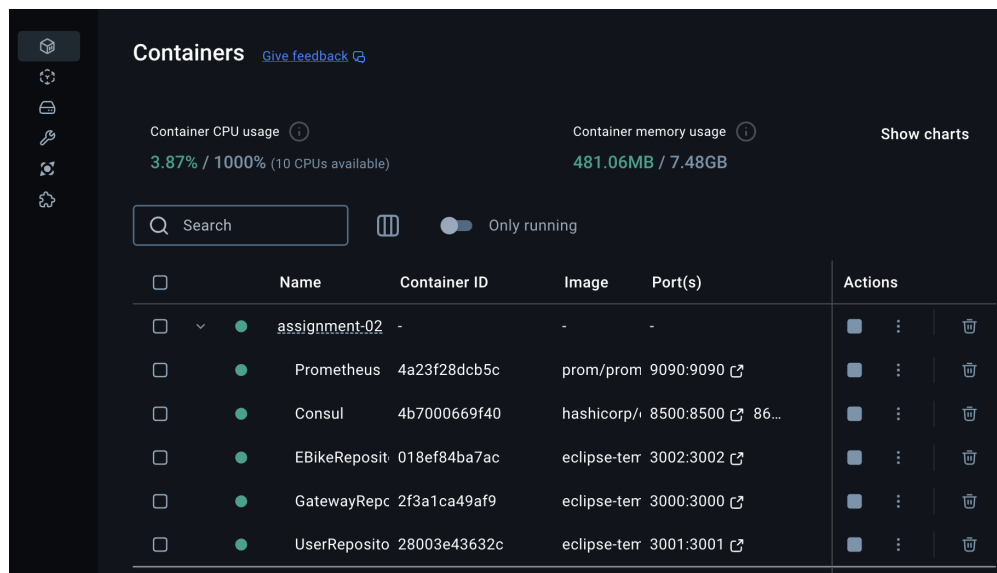
L'architettura è composta da diversi microservizi, tra cui: UserRepository, GatewayRepository, eBikeRepository e UserApplication. Ognuno di questi servizi è stato sviluppato e deployato come un container Docker indipendente, garantendo isolamento e scalabilità. Docker facilita la gestione di questi microservizi permettendo loro di essere sviluppati, testati e deployati in maniera consistente su diversi ambienti.

Deployment Pattern

Il deployment di questi microservizi segue un pattern basato su container, dove ogni microservizio è incapsulato in un container Docker. Questo approccio non solo minimizza le dipendenze tra i servizi ma permette anche una facile scalabilità orizzontale e una gestione efficiente delle risorse. Il GatewayRepository funge da punto di ingresso per le richieste esterne, indirizzandole ai servizi appropriati come UserRepository o eBikeRepository. Anche per quanto riguarda il Discovery Service, è stato deployato come un container Docker separato, garantendo che i microservizi possano registrarsi e scoprire dinamicamente gli altri servizi all'interno dell'architettura. Questo vale anche per il servizio di monitoraggio, che è stato deployato come un container Docker separato per garantire la monitoraggio continuo della salute dei servizi. Questo schema di routing è cruciale per bilanciare il carico e garantire la resilienza del sistema.

Vantaggi del Deployment con Docker

L'utilizzo di Docker per il deployment dei microservizi offre numerosi vantaggi, inclusa la portabilità. I container Docker possono essere eseguiti su quasi ogni piattaforma, riducendo significativamente i problemi legati alla discrepanza degli ambienti di sviluppo e produzione. Inoltre, Docker semplifica il processo di integrazione continua e delivery continua (CI/CD), essenziale per il mantenimento e l'aggiornamento frequente dei microservizi senza downtime significativo.



5.3 Observability Patterns

L'adozione di un approccio sistematico all'osservabilità è fondamentale per monitorare e gestire architetture software complesse, in particolare quelle basate su microservizi. Questo documento esplora l'implementazione dell'observability pattern tramite l'uso di Prometheus, una piattaforma open-source per il monitoring e l'alerting che si è affermata come uno degli strumenti più efficaci e affidabili in questo ambito.

Introduzione a Prometheus

Prometheus è stato scelto per la sua robusta architettura, la sua capacità di raccogliere metriche in tempo reale e la sua facilità di integrazione con vari ambienti di runtime. Utilizza un modello di dati multidimensionale e Query Language propria, PromQL, che permette una flessibile elaborazione delle query sulle metriche raccolte, rendendolo particolarmente adatto al monitoraggio di architetture complesse.

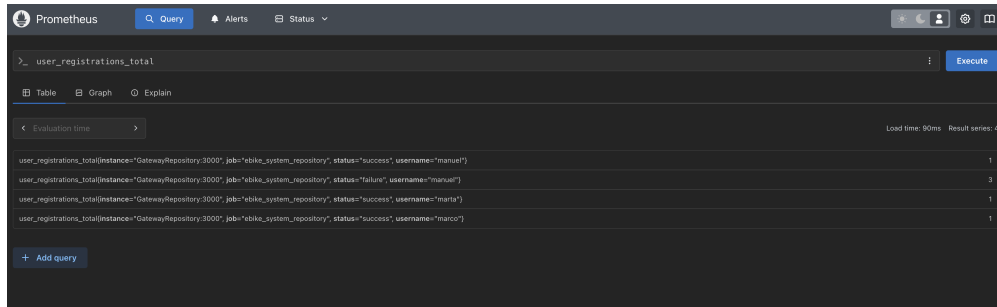
Architettura e Metriche

Nell'ambito del deployment dei microservizi, Prometheus raccoglie dati da vari nodi e servizi, utilizzando exporter specifici che espongono le metriche di interesse. Queste metriche possono includere dati su latenza, traffico, errori e saturazione dei servizi, fornendo una visione complessiva dello stato

del sistema. Prometheus supporta anche il discovery dei servizi in ambiente di produzione, facilitando la configurazione e riducendo la necessità di interventi manuali per aggiungere o rimuovere nodi dal monitoraggio.

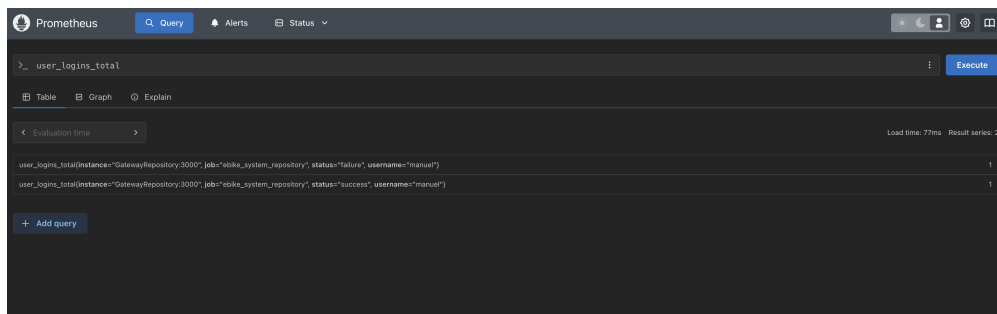
Modalità di Implementazione

Prometheus è stato utilizzato per monitorare i microservizi degli utenti e delle ebike, raccogliendo metriche relative alla latenza delle richieste, al traffico di rete e alla disponibilità dei servizi. Queste metriche sono state esposte tramite endpoint specifici, che sono stati configurati come target di scraping da parte di Prometheus.



The screenshot shows the Prometheus Query Editor interface. The query bar contains `>_ user_registrations_total`. Below the query bar, there are tabs for 'Table', 'Graph', and 'Explain'. The 'Table' tab is selected, showing a table with 4 rows of data. The table has columns for labels and values. The data is as follows:

Labels	Value
<code>user_registrations_total{instance="GatewayRepository:3000", job="ebike_system_repository", status="success", username="manuel"}</code>	1
<code>user_registrations_total{instance="GatewayRepository:3000", job="ebike_system_repository", status="failure", username="manuel"}</code>	3
<code>user_registrations_total{instance="GatewayRepository:3000", job="ebike_system_repository", status="success", username="maria"}</code>	1
<code>user_registrations_total{instance="GatewayRepository:3000", job="ebike_system_repository", status="success", username="marco"}</code>	1



The screenshot shows the Prometheus Query Editor interface. The query bar contains `>_ user_logins_total`. Below the query bar, there are tabs for 'Table', 'Graph', and 'Explain'. The 'Table' tab is selected, showing a table with 2 rows of data. The table has columns for labels and values. The data is as follows:

Labels	Value
<code>user_logins_total{instance="GatewayRepository:3000", job="ebike_system_repository", status="failure", username="manuel"}</code>	1
<code>user_logins_total{instance="GatewayRepository:3000", job="ebike_system_repository", status="success", username="manuel"}</code>	1

5.4 Runtime Configuration Patterns

Il runtime configuration pattern è un modello di progettazione software che permette agli sviluppatori di modificare il comportamento di un'applicazione al momento dell'esecuzione senza dover intervenire direttamente sul codice sorgente. Questo approccio è particolarmente utile in ambienti distribuiti e scalabili, dove la flessibilità e l'adattabilità sono essenziali.

Configurazione di UserRepository e eBikeRepository

Nel contesto specifico, il file `config.property` è stato utilizzato per configurare il microservizio UserRepository, impostando la porta di comunicazione e definendo gli utenti di default. Questa configurazione permette di adattare facilmente il servizio a diversi ambienti di deployment o di rispondere rapidamente a requisiti aziendali che cambiano.

Analogamente, lo stesso tipo di file di configurazione è stato impiegato per il microservizio eBikeRepository. Qui, oltre alla porta, il file `config.property` specifica le eBike di default disponibili

al lancio del servizio. Questo assicura che il servizio possa essere configurato rapidamente a seconda delle necessità senza necessità di modifiche profonde al codice o alla logica del programma.

Vantaggi dell'Uso del Runtime Configuration Pattern

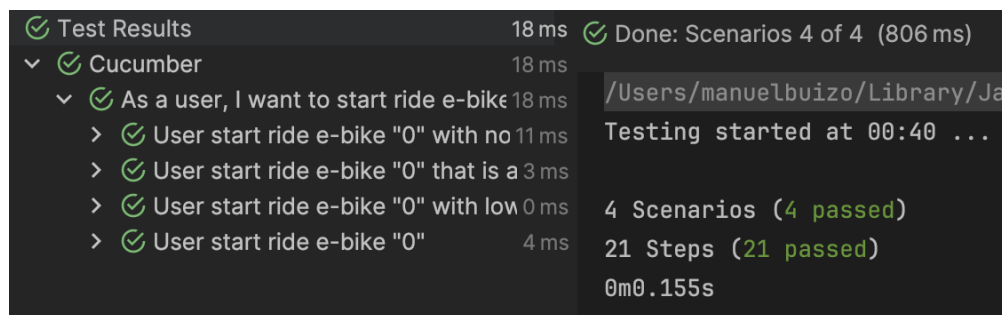
1. **Flessibilità:** Il pattern di configurazione a runtime consente agli amministratori di sistema e agli sviluppatori di modificare le impostazioni operative senza il bisogno di rilasciare nuove versioni del software. Questo risulta in una maggiore agilità operativa e una riduzione dei tempi di inattività per manutenzione.
2. **Separazione tra codice e configurazione:** Mantenendo separate le configurazioni dall'applicazione vera e propria, si aumenta la pulizia del codice e si riducono gli errori derivanti dalla gestione diretta delle variabili operative nel codice sorgente.
3. **Scalabilità:** Con la configurazione esternalizzata, è più semplice scalare l'applicazione aggiungendo nuove istanze dei servizi configurati senza modificare ogni singola implementazione. Ogni istanza può leggere la propria configurazione all'avvio, adattandosi dinamicamente al contesto di esecuzione.
4. **Manutenzione semplificata:** Aggiornamenti, modifiche e troubleshooting possono essere gestiti modificando i file di configurazione, senza necessità di interventi diretti sul codice, facilitando così anche il lavoro di manutenzione e supporto.

5.5 Testing patterns

Il testing è un elemento cruciale nello sviluppo di software per garantire la qualità e la funzionalità dei sistemi. In un ambiente complesso basato su microservizi come quello composto da UserRepository e eBikeRepository, l'adozione di strategie di testing efficaci è essenziale. Questo documento descrive l'approccio adottato, focalizzandosi sull'utilizzo di Cucumber per i test basati su scenari e Postman per il testing delle API REST.

Utilizzo di Cucumber per i Test Basati su Scenari

Cucumber è uno strumento di testing che supporta la metodologia Behaviour-Driven Development (BDD). Permette agli sviluppatori di definire test in un linguaggio naturale, rendendo i casi di test comprensibili anche per i non tecnici. Nel progetto corrente, Cucumber è stato utilizzato per definire e eseguire vari scenari nel dominio degli user e delle eBike. Questo approccio ha facilitato la collaborazione tra sviluppatori, tester e stakeholder, migliorando la definizione delle specifiche e assicurando che le funzionalità sviluppate rispondano effettivamente alle esigenze del business.



```
✓ Test Results 18 ms ✓ Done: Scenarios 4 of 4 (806 ms)
└─ ✓ Cucumber 18 ms
   └─ ✓ As a user, I want to start ride e-bike 18 ms
      > ✓ User start ride e-bike "0" with no 11 ms
      > ✓ User start ride e-bike "0" that is a 3 ms
      > ✓ User start ride e-bike "0" with low 0 ms
      > ✓ User start ride e-bike "0" 4 ms

/Users/manuelbuizo/Library/Ja
Testing started at 00:40 ...

4 Scenarios (4 passed)
21 Steps (21 passed)
0m0.155s
```

Utilizzo di Postman per il Testing delle API REST

Postman, uno strumento ampiamente utilizzato per il testing di API, è stato impiegato per testare le API REST create per UserRepository e eBikeRepository. Grazie a Postman, è stato possibile inviare richieste HTTP ai microservizi, verificando la correttezza delle risposte in vari scenari, compreso il controllo dei codici di stato, delle risposte JSON, e della gestione degli errori. Questo ha permesso di assicurare che le interfacce API funzionino correttamente e conformemente alle specifiche, facilitando ulteriormente l'integrazione tra i diversi componenti del sistema.

Vantaggi nell'Uso di Questi Strumenti

1. **Clarity and Collaboration:** L'uso di Cucumber migliora la comunicazione tra i membri del team e gli stakeholder, essendo i test scritti in un linguaggio naturale. Questo aiuta tutti i partecipanti al progetto a comprendere meglio i requisiti e gli scenari di testing.
2. **Immediate Feedback:** Postman permette di testare rapidamente e in modo interattivo le API, offrendo un feedback immediato. Questo è cruciale per identificare e risolvere rapidamente i problemi durante le fasi di sviluppo.

3. Automazione e Integrazione: Entrambi gli strumenti supportano l'automazione dei test. Cucumber può essere integrato con altri strumenti come Selenium per test di accettazione automatizzati, mentre Postman può essere utilizzato con sistemi di integrazione continua per eseguire test API automatici.

